

Java 11 試題

20 題答案與逐題詳解

範圍：primitive type、operator、if、loop、field、constructors、methods、overload、static、一維 array

題型：單選題＋複選題 | JDK 11 | 原創模擬題（非 Oracle 真題）

答案速查

題號	答案	題號	答案
1	C	11	B
2	B	12	D
3	B、C	13	B、C、E
4	C	14	A、B、E
5	C	15	C
6	B	16	B、D、E
7	C、E	17	C
8	C	18	A、B、C
9	C	19	D
10	B	20	A、E

第 1 題 | 單選題（選出 1 個答案）

執行下列程式後，輸出結果為何？

```
public class Test1 {  
    public static void main(String[] args) {  
        int a = 3;  
        int b = a++ + ++a * 2;  
        System.out.println(a + ":" + b);  
    }  
}
```

- A. 4:11
- B. 5:11
- C. 5:13
- D. 6:13
- E. 程式無法編譯

正確答案：C

核心解析：Java 由左到右計算運算元。a++ 先提供 3，再把 a 變成 4；接著 ++a 先把 a 變成 5，再提供 5， $5 * 2 = 10$ 。因此 $b = 3 + 10 = 13$ ，最後 $a = 5$ 。

選項	詳解
A	錯。忽略了兩次遞增造成的最終值與乘法結果。
B	錯。a 最後確實為 5，但 b 不是 11。
C	對。最終輸出 5:13。
D	錯。a 只增加兩次，最後為 5。
E	錯。所有型別與運算皆合法。

第 2 題 | 單選題（選出 1 個答案）

下列程式的執行結果為何？

```
public class Test2 {  
    public static void main(String[] args) {  
        byte b = 10;  
        b += 130;  
        System.out.println(b);  
    }  
}
```

- A. 140
- B. -116
- C. -126
- D. 編譯失敗，因為 130 超過 byte 範圍
- E. 執行時發生錯誤

正確答案：B

核心解析：複合指定運算 $b += 130$ 會隱含型別轉換，等價概念為 $b = (\text{byte})(b + 130)$ 。 $10 + 130 = 140$ ，轉為 byte 後發生溢位， $140 - 256 = -116$ 。

選項	詳解
A	錯。140 無法直接以 byte 表示。
B	對。byte 溢位後為 -116。
C	錯。溢位計算不正確。
D	錯。+= 具有隱含轉型，因此可編譯。
E	錯。這是合法的整數溢位，不會丟出例外。

第 3 題 | 複選題（選出 2 個答案）

假設 `int x = 0;` 已宣告。下列哪些 if 敘述可以成功編譯？

- A. `if (x = 5) { }`
- B. `if ((x = 5) > 3) { }`
- C. `if (x == 5) { }`
- D. `if (x => 5) { }`
- E. `if (5 < x < 10) { }`

正確答案：B、C

核心解析：Java 的 if 條件必須是 boolean。B 的指定運算先把 5 指定給 x，再比較 `> 3`，結果為 boolean；C 的 `==` 比較也回傳 boolean。

選項	詳解
A	錯。x = 5 的結果型別是 int，不能當作 if 條件。
B	對。(x = 5) > 3 的結果是 boolean。
C	對。x == 5 回傳 boolean。
D	錯。Java 沒有 => 運算子。
E	錯。Java 不支援數學式的連鎖比較；(5 < x) 先得到 boolean，不能再與 10 比較。

第 4 題 | 單選題（選出 1 個答案）

執行下列程式後，輸出結果為何？

<pre>public class Test4 { public static void main(String[] args) { int x = 4; int y = 2; if (x++ > 4 && ++y > 2) { System.out.print("A"); } System.out.println(x + ":" + y); } }</pre>

- A. A:3
- B. 4:2
- C. 5:2

D. 5:3

E. 程式無法編譯

正確答案：C

核心解析：`x++ > 4` 先以 4 參與比較，條件為 `false`，但 `x` 仍增加為 5。`&&` 具有 `short-circuit`，因此右側 `++y > 2` 不執行，`y` 保持 2。`if` 區塊也不執行。

選項	詳解
A	錯。第一個條件為 <code>false</code> ，A 不會輸出，右側也不執行。
B	錯。 <code>x++</code> 仍會讓 <code>x</code> 增加為 5。
C	對。輸出 5:2。
D	錯。短路使 <code>y</code> 不會增加。
E	錯。程式可正常編譯。

第 5 題 | 單選題（選出 1 個答案）

下列程式輸出為何？

```
public class Test5 {
    public static void main(String[] args) {
        int sum = 0;
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue;
            }
            sum += i;
        }
        System.out.println(sum);
    }
}
```

A. 9

B. 10

C. 12

D. 15

E. 程式無法編譯

正確答案：C

核心解析：`i = 3` 時 `continue` 跳過本次剩餘敘述，因此 `sum` 加入 1、2、4、5，共 12。

選項	詳解
A	錯。漏算部分迭代。
B	錯。不是只加 1 到 4。
C	對。 $1 + 2 + 4 + 5 = 12$ 。
D	錯。15 是沒有 <code>continue</code> 時的總和。
E	錯。 <code>continue</code> 在迴圈中合法。

第 6 題 | 單選題（選出 1 個答案）

下列巢狀迴圈執行後，count 的值為何？

```
public class Test6 {  
    public static void main(String[] args) {  
        int count = 0;  
  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= i; j++) {  
                if (j == 2) {  
                    break;  
                }  
                count++;  
            }  
        }  
        System.out.println(count);  
    }  
}
```

- A. 2
- B. 3
- C. 4
- D. 6
- E. 無限迴圈

正確答案：B

核心解析：break 只離開內層迴圈。i=1 時 count 加 1；i=2 時只有 j=1 加 1，j=2 break；i=3 亦只有 j=1 加 1。總計 3。

選項	詳解
A	錯。三次外層迭代都至少執行一次 j=1。
B	對。總共執行三次 count++。
C	錯。j=2 不會執行 count++。
D	錯。break 阻止內層完整執行。
E	錯。兩個 for 都有正常終止條件。

第 7 題 | 複選題（選出 2 個答案）

關於下列程式，哪些敘述正確？

```
public class Test7 {  
    public static void main(String[] args) {  
        int x = 3;  
        do {  
            System.out.print(x);  
            x--;  
        } while (x > 3);  
        System.out.print(":" + x);  
    }  
}
```

- A. 程式無法編譯，因為 do-while 的條件第一次就是 false
- B. 迴圈本體完全不會執行

- C. 第一段輸出為 3
- D. 最終 x 的值為 3
- E. 完整輸出為 3:2

正確答案：C、E

核心解析：do-while 至少執行一次。本體先輸出 3，再 x-- 變成 2；條件 2 > 3 為 false，因此結束，最後輸出 :2。

選項	詳解
A	錯。do-while 會先執行本體，再判斷條件，因此可正常編譯與執行。
B	錯。do-while 至少執行一次。
C	對。第一次輸出 3。
D	錯。執行 x-- 後為 2。
E	對。完整輸出 3:2。

第 8 題 | 單選題（選出 1 個答案）

執行下列程式後，輸出結果為何？

<pre> class Account { int id; double balance; boolean active; void show() { System.out.println(id + ":" + balance + ":" + active); } public static void main(String[] args) { Account a = new Account(); a.show(); } } </pre>

- A. null:null:false
- B. 0:0:false
- C. 0:0.0:false
- D. 程式無法編譯，因為 field 未初始化
- E. 執行時發生錯誤

正確答案：C

核心解析：instance field 具有預設值：int 為 0、double 為 0.0、boolean 為 false。

選項	詳解
A	錯。primitive field 不使用 null 作預設值。
B	錯。double 顯示為 0.0。
C	對。輸出 0:0.0:false。
D	錯。field 會自動取得預設值。
E	錯。程式可正常執行。

第 9 題 | 單選題（選出 1 個答案）

下列程式的結果為何？

```
class Item {
    int price;

    Item(int price) {
        this.price = price;
    }

    public static void main(String[] args) {
        Item a = new Item();
        System.out.println(a.price);
    }
}
```

- A. 輸出 0
- B. 輸出 null
- C. 編譯失敗，因為找不到無參數 constructor
- D. 編譯成功，但執行時發生錯誤
- E. 編譯失敗，因為 constructor 必須宣告 void

正確答案：C

核心解析：類別已明確宣告 `Item(int price)`，因此編譯器不會再提供預設的無參數 constructor。`new Item()` 找不到相符 constructor。

選項	詳解
A	錯。物件根本無法建立。
B	錯。price 是 int，不會是 null。
C	對。沒有 <code>Item()</code> 。
D	錯。錯誤發生在編譯期。
E	錯。constructor 不可宣告回傳型別，包括 void。

第 10 題 | 單選題（選出 1 個答案）

執行下列程式後輸出為何？

```
class Product {
    int price = 100;

    Product(int price) {
        price = price;
    }

    void show() {
        System.out.println(price);
    }

    public static void main(String[] args) {
        new Product(500).show();
    }
}
```

- A. 0
- B. 100
- C. 500
- D. 編譯失敗，因為 price 名稱重複
- E. 執行時發生錯誤

正確答案：B

核心解析：constructor 參數 price 遮蔽 field。price = price 是把參數指定給自己，沒有修改 this.price，因此 field 保持初始化值 100。

選項	詳解
A	錯。field 已明確初始化為 100。
B	對。this.price 未被改變。
C	錯。若要得到 500，應寫 this.price = price。
D	錯。參數與 field 同名是合法的 shadowing。
E	錯。程式可正常執行。

第 11 題 | 單選題（選出 1 個答案）

下列 overload 程式輸出為何？

```
public class Test11 {
    static void print(int x) {
        System.out.print("I");
    }

    static void print(long x) {
        System.out.print("L");
    }

    static void print(double x) {
        System.out.print("D");
    }

    public static void main(String[] args) {
        byte b = 1;
        print(b);
        print(1L);
        print(1.0f);
    }
}
```

- A. IID
- B. ILD
- C. LLD
- D. IDD
- E. 編譯失敗，因為 byte 呼叫不明確

正確答案：B

核心解析：byte 可 widening 到 int，且 int 是最接近的可用 overload；1L 精確符合 long；float 可 widening 到 double。因此依序輸出 I、L、D。

選項	詳解
A	錯。1L 應選 long 版本。
B	對。輸出 ILD。
C	錯。byte 優先 widening 到 int，不會跳到 long。
D	錯。1L 不會選 double。
E	錯。byte 到 int 是明確且較近的 widening。

第 12 題 | 單選題（選出 1 個答案）

下列程式會發生什麼結果？

```
public class Test12 {
    static void test(int a, long b) {
        System.out.print("A");
    }

    static void test(long a, int b) {
        System.out.print("B");
    }

    public static void main(String[] args) {
        test(10, 10);
    }
}
```

- A. 輸出 A
- B. 輸出 B
- C. 輸出 AB
- D. 編譯失敗，因為 method call 不明確
- E. 執行時才決定呼叫哪個 method

正確答案：D

核心解析：兩個 overload 都需要一次 int→long widening，且沒有任何一個方法在兩個參數上都比另一個更具體，因此呼叫 test(10, 10) 產生 ambiguity。

選項	詳解
A	錯。第一個 overload 不比第二個更具體。
B	錯。第二個也不比第一個更具體。
C	錯。一次 method call 只會選一個方法。
D	對。編譯期即判定 ambiguous。
E	錯。overload 在編譯期解析。

第 13 題 | 複選題（選出 3 個答案）

類別中已存在 void work(int x) { }。下列哪些方法可以再宣告於同一類別中，而形成合法 overload？

- A. int work(int x) { return x; }
- B. void work(long x) { }
- C. void work(int x, int y) { }

D. static void work(int x) { }

E. void work(double x) { }

正確答案：B、C、E

核心解析：method overload 必須改變參數列表（數量或型別）。只改回傳型別或只加 static 都不會改變 method signature。

選項	詳解
A	錯。只有回傳型別不同，signature 仍是 work(int)。
B	對。參數型別改為 long。
C	對。參數數量不同。
D	錯。static 不屬於 overload signature，仍與 work(int) 衝突。
E	對。參數型別改為 double。

第 14 題 | 複選題（選出 3 個答案）

類別中已存在 constructor Order(int x) { }。下列哪些 constructor 可以再加入而形成合法 constructor overload？

A. Order() { }

B. Order(long x) { }

C. Order(int y) { }

D. static Order(int x) { }

E. Order(int x, int y) { }

正確答案：A、B、E

核心解析：constructor overload 與 method overload 相同，必須有不同參數列表。constructor 不能使用 static，也不能只靠參數名稱不同來區分。

選項	詳解
A	對。參數數量不同。
B	對。參數型別不同。
C	錯。參數名稱不影響 signature，仍是 Order(int)。
D	錯。constructor 不能宣告 static，且語法本身非法。
E	對。參數數量不同。

第 15 題 | 單選題（選出 1 個答案）

執行下列程式後輸出為何？

```
class User {
    static int count = 0;
    int id;

    User(int id) {
        this.id = id;
        count++;
    }

    public static void main(String[] args) {
        new User(1);
        new User(2);
        User u3 = new User(3);
        System.out.println(User.count + ":" + u3.count);
    }
}
```

- A. 1:1
- B. 3:1
- C. 3:3
- D. 0:0
- E. 編譯失敗，static field 不能透過物件存取

正確答案：C

核心解析：count 是類別共用的 static field，每建立一個 User 就加 1，因此建立三個物件後 count = 3。Java 允許透過物件參考存取 static 成員（雖然不建議），所以 u3.count 也看到同一個 3。

選項	詳解
A	錯。count 不屬於單一物件。
B	錯。u3.count 仍指向同一個 static field。
C	對。輸出 3:3。
D	錯。constructor 已執行三次 count++。
E	錯。語法允許，但建議使用 User.count。

第 16 題 | 複選題（選出 3 個答案）

考慮下列類別。若把選項中的敘述分別放入 static void test() 方法內，哪些可以成功編譯？

```
class Calc {
    int a = 10;
    static int b = 20;

    void m1() { }
    static void m2() { }

    static void test() {
        // 將各選項分別放在這裡測試
    }
}
```

- A. System.out.println(a);

- B. System.out.println(b);
- C. m1();
- D. m2();
- E. new Calc().a = b;

正確答案：B、D、E

核心解析：static context 無法直接存取 instance field 或 instance method，但可直接存取 static field/method；若先建立物件，則可經由物件存取 instance field。

選項	詳解
A	錯。a 是 instance field，沒有物件參考。
B	對。b 是 static field。
C	錯。m1() 是 instance method。
D	對。m2() 是 static method。
E	對。new Calc() 建立物件後可存取該物件的 a。

第 17 題 | 單選題（選出 1 個答案）

下列程式輸出為何？

```
public class Test17 {  
    public static void main(String[] args) {  
        int[] a = {2, 4, 6, 8};  
        int x = a[1] + a[a.length - 1];  
        System.out.println(x);  
    }  
}
```

- A. 8
- B. 10
- C. 12
- D. 14
- E. 發生 ArrayIndexOutOfBoundsException

正確答案：C

核心解析：a[1] = 4，a.length = 4，因此 a[a.length - 1] = a[3] = 8，總和為 12。

選項	詳解
A	錯。只取到最後一個元素。
B	錯。索引計算不正確。
C	對。4 + 8 = 12。
D	錯。不是 6 + 8。
E	錯。最後合法索引就是 length - 1。

第 18 題 | 複選題（選出 3 個答案）

下列哪些一維 int array 宣告與建立方式可以成功編譯？

- A. `int[] a = new int[3];`
- B. `int b[] = {1, 2, 3};`
- C. `int[] c = new int[] {1, 2, 3};`
- D. `int[3] d = new int[];`
- E. `int[] e = new int[3]{1, 2, 3};`

正確答案：A、B、C

核心解析：Java 支援 `int[] a` 與 `int a[]` 兩種宣告形式，也可使用 `new int[] {...}`。指定 `new int[3]` 時不能再接 `initializer`；型別宣告中也不能寫 `int[3]`。

選項	詳解
A	對。建立長度 3、預設值皆為 0 的陣列。
B	對。合法的宣告與 <code>initializer</code> 。
C	對。匿名陣列建立語法合法。
D	錯。型別宣告位置不可寫長度，而且 <code>new int[]</code> 缺少 <code>initializer</code> 。
E	錯。 <code>new int[3]</code> 與 <code>{1,2,3}</code> 不能同時指定。

第 19 題 | 單選題（選出 1 個答案）

執行下列程式後輸出為何？

```
public class Test19 {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4};

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] % 2 == 0) {
                nums[i] *= 2;
            } else {
                nums[i] += 1;
            }
        }

        System.out.println(nums[0] + nums[1] + nums[2] + nums[3]);
    }
}
```

- A. 10
- B. 14
- C. 16
- D. 18
- E. 20

正確答案：D

核心解析：每個元素都依原值判斷一次：1→2、2→4、3→4、4→8，最後總和 2 + 4 + 4 + 8 = 18。

選項	詳解
A	錯。10 是原始總和。
B	錯。未正確套用所有修改。
C	錯。偶數元素會乘 2。
D	對。結果為 18。
E	錯。沒有任何元素重複處理。

第 20 題 | 複選題（選出 2 個答案）

關於下列程式，哪些敘述正確？

```
class Sale {
    static int total;
    int amount;

    Sale(int amount) {
        this.amount = amount;
        if (amount > 100) {
            total += bonus(amount);
        } else {
            total += amount;
        }
    }

    static int bonus(int amount) {
        return amount + 10;
    }

    static int bonus(long amount) {
        return (int) amount + 20;
    }

    public static void main(String[] args) {
        int[] data = {100, 101, 102};
        for (int i = 0; i < data.length; i++) {
            new Sale(data[i]);
        }
        System.out.println(Sale.total);
    }
}
```

- A. 程式可成功編譯並輸出 323
- B. `bonus(data[i])` 會選擇 `bonus(long)`
- C. 若把 `bonus(amount)` 改成 `bonus((long) amount)`，最後輸出仍然是 323
- D. 每個 `Sale` 物件各有自己獨立的 `total`
- E. `static method bonus(int)` 可以被 `constructor` 直接呼叫

正確答案：A、E

核心解析：`data` 元素型別為 `int`，`constructor` 的 `amount` 也是 `int`，因此 `bonus(amount)` 精確匹配 `bonus(int)`。累計為 $100 + (101+10) + (102+10) = 323$ 。`total` 是 `static`，所有物件共用；`constructor` 可直接呼叫同類別的 `static method`。

選項	詳解
A	對。總計 323。
B	錯。 <code>int</code> 精確匹配 <code>bonus(int)</code> ，不會 widening 到 <code>long</code> 。

選項	詳解
C	錯。強制轉成 long 後會選 bonus(long)，總計變成 $100 + 121 + 122 = 343$ 。
D	錯。static total 屬於類別共用。
E	對。constructor 可以直接呼叫同類別 static method。